



**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

Monitorização de Sensores IoT

PARADIGMAS EMERGENTES PARA O DESENVOLVIMENTO WEB E MOBILE

MESTRADO EM ENGENHARIA INFORMÁTICA

Grupo 1

DIOGO PEREIRA – 8200594

HUGO GUIMARÃES – 8220337

29 DE MAIO DE 2026

Conteúdo

1	Introdução	5
1.1	Contextualização e Tema do Projeto	5
1.2	Objetivos do Trabalho	5
1.3	Estrutura do Relatório	6
2	Tecnologias Utilizadas	7
2.1	Flutter (Front-end Web e Mobile)	7
2.2	Scala e ZIO (Back-end)	7
2.3	C++ (Simulação de Sensores IoT)	8
3	Arquitetura da Solução	9
3.1	Camada de Simulação (Ingestão de Dados)	9
3.2	<i>Back-end</i> (Processamento e Persistência)	10
3.3	<i>Front-end</i> (Visualização Web e Mobile)	10
3.4	Fluxo de Dados e Interação	10
4	Paradigmas Implementados	13
4.1	Programação Funcional	13
4.2	Programação Orientada a Aspectos	14
4.3	Programação Orientada a Eventos	15
5	Interface de Utilizador	17
5.1	Ecrã Inicial	17
5.2	Ecrã para inserção de novos sensores	18
5.3	Gráfico histórico de medições	18
5.4	Alterar mínimos e máximos	19
6	Metodologia de Avaliação e Desempenho Individual	21
6.1	Critérios de Atribuição de <i>Story Points</i>	21
6.2	Distribuição do Esforço e Contribuição	21
6.2.1	Contribuição: Diogo (50% – 94 SP)	21
6.2.2	Contribuição: Hugo (50% – 84 SP)	22

7 Conclusão	24
7.1 Trabalho Futuro	24

Índice de Figuras

1	Diagrama de Arquitetura de Alto Nível	9
2	Diagrama de Sequência ilustrando o fluxo de eventos em tempo real.	12
3	Ecrã inicial com as medições dos sensores em tempo real.	17
4	Interface para adição e gestão de sensores no painel.	18
5	Gráfico com o histórico temporal de medições de um sensor selecionado. . .	19
6	Dialog para edição dos limites do mostrador e remoção do sensor.	20
7	Distribuição da carga de trabalho do projeto com base em <i>Story Points</i> . . .	22

Índice de Excertos de Código

1	Processamento funcional de uma <i>Stream</i> de dados no <i>front-end</i>	14
---	---	----

1 Introdução

1.1 Contextualização e Tema do Projeto

No cenário tecnológico atual, o quotidiano das pessoas e das organizações é cada vez mais rodeado por redes de dispositivos inteligentes e sensores que recolhem dados em tempo real. Estes sensores, que podem medir variáveis como a qualidade do ar, temperatura e humidade, geram fluxos contínuos de informação que exigem sistemas robustos capazes de os processar e transformar em conhecimento útil.

Foi com base nesta necessidade que o presente projeto foi desenvolvido, assumindo-se como um elemento integrador no âmbito da unidade curricular de Paradigmas Emergentes para o Desenvolvimento Web e Mobile do Mestrado em Engenharia Informática. O trabalho propõe o desenvolvimento de uma aplicação para a gestão e monitorização de uma rede de sensores IoT (Internet of Things) simulados. Para dar resposta a este desafio e garantir uma arquitetura resiliente e reativa, a solução foi desenhada com base em três pilares fundamentais exigidos: Programação Funcional, Programação Orientada a Aspectos e Programação *Event-Driven* (Orientada a Eventos). O sistema permite a monitorização contínua destes dados em tempo real através de plataformas Web e Mobile.

1.2 Objetivos do Trabalho

Este trabalho prático tem como principal objetivo consolidar e integrar os conhecimentos adquiridos durante a unidade curricular, estimulando a exploração de conceitos avançados e tecnologias emergentes.

Para a concretização deste propósito principal, foram definidos os seguintes objetivos específicos:

- **Aplicação de Paradigmas Emergentes:** Desenvolver a arquitetura do sistema aplicando os conceitos de Programação Funcional (garantindo imutabilidade e funções puras), Programação Orientada a Aspectos (separando preocupações transversais como *logs* ou métricas) e Programação *Event-Driven* (para lidar com o fluxo de dados dos sensores).
- **Desenvolvimento Multiplataforma:** Construir a solução contendo, no mínimo, um componente *front-end* para *Web* e outro para ambiente *Mobile*, garantindo a

acessibilidade aos *dashboards* de monitorização.

- **Comunicação em Tempo Real e Eficiente:** Utilizar *WebSockets* para o envio de eventos e alertas instantâneos para os clientes, e integrar *GraphQL* para otimizar as consultas de dados, evitando *over-fetching*.
- **Gestão de Dispositivos IoT:** Capacitar a aplicação para gerir múltiplos sensores em simultâneo, processar as suas leituras e fornecer alertas em tempo real perante anomalias (e.g., aumentos súbitos de temperatura).

1.3 Estrutura do Relatório

O presente relatório está organizado de forma a detalhar todo o processo de conceção e desenvolvimento da solução. A Secção 2 apresenta as tecnologias escolhidas para o *front-end* e *back-end*. A Secção 3 expõe a arquitetura da solução, acompanhada do respetivo diagrama arquitetural e dos diagramas UML necessários para o seu entendimento técnico. A Secção 4 detalha o mapeamento dos objetivos com o trabalho desenvolvido, nomeadamente a forma como os paradigmas Funcional, Orientado a Aspectos e *Event-Driven* foram implementados no código. A Secção 5 foca-se na demonstração das interfaces concebidas (Web e Mobile) e na exploração das suas funcionalidades. A Secção 6 apresenta um gráfico detalhado com os *story points* atribuídos e concluídos ao longo do projeto, evidenciando o desempenho e esforço individual de cada estudante. Por fim, a Secção 7 realiza uma breve súmula do trabalho, enumerando as conclusões retiradas e possíveis linhas de trabalho futuro.

2 Tecnologias Utilizadas

2.1 Flutter (Front-end Web e Mobile)

O motivo por detrás da escolha do Flutter para a realização deste projeto reside na flexibilidade do mesmo de poder ser facilmente compilado para diversas plataformas. A solução foi estruturada em duas aplicações: um *dashboard Web* e aplicação *Mobile*. Para evitar a duplicação de esforço e garantir a consistência, ambas as partes estão unidas por um *package* interno partilhado, que centraliza as entidades, a gestão de estado e toda a lógica com o *back-end*.

Outro dos motivos para a escolha desta tecnologia foi a alta performance para renderizar o conteúdo no ecrã. Ao desenhar cada pixel diretamente no ecrã de forma independente do sistema operativo, o Flutter permite que gráficos de monitorização dos sensores e sejam executados de forma fluida, tanto num navegador *Web* como num dispositivo *Mobile* com recursos mais limitados.

2.2 Scala e ZIO (Back-end)

O Scala em conjunto com o ecossistema ZIO foi adotado de modo a cumprir os requisitos arquiteturais impostos para este projeto, nomeadamente a aplicação dos três paradigmas emergentes. Em primeiro lugar, a linguagem Scala fomenta nativamente a programação funcional, garantindo que a validação e transformação dos dados dos sensores é feita através de funções puras e estruturas de dados imutáveis, eliminando efeitos secundários indesejados.

Em segundo lugar, a biblioteca ZIO oferece abstrações como as *ZStreams*, que foram essenciais para implementar a programação orientada a eventos. Em vez de um modelo de pedido-resposta tradicional, o sistema processa um fluxo contínuo de leituras (*pipeline* de eventos) proveniente dos sensores, filtrando anomalias e publicando alertas de forma reativa.

Por fim, o ZIO suporta nativamente a programação orientada a aspetos através de *ZIO Aspects*. Isto permitiu isolar preocupações transversais, como o registo de *logs* de todas as leituras e medição de métricas de latência. Assim, o código permanece limpo e focado apenas no processamento da telemetria, enquanto os aspetos são aplicados de forma declarativa e invisível. A alta concorrência é garantida através das *Fibers* do ZIO,

que, à semelhança das *Goroutines*, são bastante leves e permitem ao servidor suportar milhares de eventos concorrentes com um consumo mínimo de memória.

2.3 C++ (Simulação de Sensores IoT)

Embora os sensores deste projeto sejam simulados, a escolha da linguagem C++ foi deliberada para manter a máxima fidelidade com os ambientes de Internet of Things (IoT) do mundo real. Na indústria, a maior parte dos microcontroladores (como o ecossistema Arduino ou o ESP32) é programada em C ou C++ devido ao controlo de baixo nível sobre a memória e o *hardware*.

Ao desenvolver os simuladores em C++, o projeto mimetiza o comportamento, as restrições e o formato de dados que um sensor de temperatura, humidade ou qualidade do ar real emitiria. Estes simuladores atuam como os produtores iniciais no fluxo de dados, gerando eventos a uma taxa configurável que alimentam o *broker* de mensagens e, consequentemente, todo o *pipeline* reativo do *back-end*.

3 Arquitetura da Solução

O sistema está representado na Figura 1, encontra-se dividido em três camadas: Simulador, *Back-end* e *Front-end*. O fluxo de informação ocorre de forma descendente no que toca à telemetria, e de forma bidirecional na interação com o cliente.

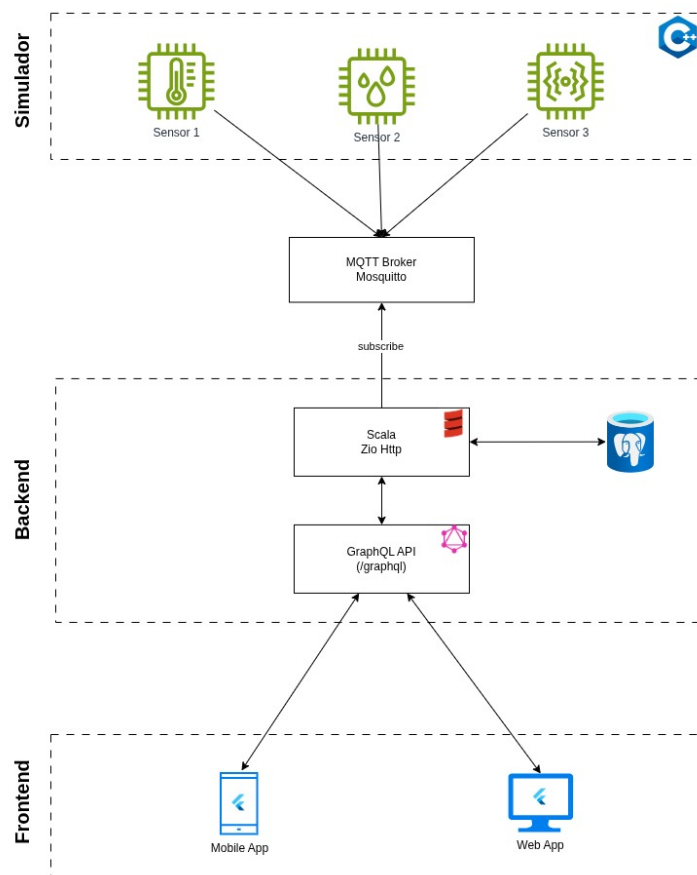


Figura 1: Diagrama de Arquitetura de Alto Nível

3.1 Camada de Simulação (Ingestão de Dados)

Na camada superior encontram-se os componentes desenvolvidos em C++, responsáveis por mimetizar o comportamento de dispositivos físicos num ambiente IoT (Sensores 1, 2 e 3). Estes simuladores geram medições contínuas (como temperatura e humidade) e atuam como produtores (*publishers*). Os dados gerados são enviados para um *Broker* MQTT (neste caso, o *Mosquitto*) seguindo a especificação Sparkplug B. Esta convenção tira partido de *Protocol Buffers* (Protobuf) para a serialização binária dos dados, um padrão que reduz drasticamente o tamanho do *payload* e acelera a comunicação. A escolha do MQTT e do Sparkplug B justifica-se por constituir o ecossistema padrão e mais eficiente

na comunicação *machine-to-machine* (M2M) na indústria.

3.2 *Back-end* (Processamento e Persistência)

O Back-end, desenvolvido em Scala com recurso à biblioteca *ZIO* e *ZIO HTTP*. O fluxo de operações decorre da seguinte forma:

- **Subscrição e Processamento:** O servidor Scala atua como cliente do *broker* MQTT, subcrevendo os tópicos dos sensores. À medida que as mensagens chegam, entram numa *pipeline*, onde a informação é validada e filtrada.
- **Armazenamento de Histórico:** O *back-end* comunica com uma base de dados relacional PostgreSQL para persistir os dados processados.
- **Interface de Comunicação:** Para servir as aplicações cliente, o servidor expõe uma API GraphQL.

3.3 *Front-end* (Visualização Web e Mobile)

Na camada de *Front-end* encontram-se a aplicação para dispositivos móveis e a aplicação *Web*, ambas desenvolvidas na *framework* Flutter a partir de um *package* partilhado entre as duas, evitando duplicação desnecessária de código.

Estas aplicações comunicam exclusivamente com o serviço de *Back-end* através da API GraphQL. Esta ligação reflete-se na arquitetura através das setas bidirecionais, pois o cliente não realiza apenas *Queries* (pedidos HTTP) para obter o histórico guardado no PostgreSQL. Crucialmente, as aplicações cliente estabelecem ligações persistentes via *WebSockets* (através de *Subscriptions* GraphQL) com o servidor Scala. Isto garante que sempre que um novo evento passa pela *pipeline* do *back-end*, os *dashboards* Flutter são atualizados em tempo real, sem necessidade de *polling* constante.

3.4 Fluxo de Dados e Interação

Todo o fluxo de dados da aplicação está descrito na Figura 2, que demonstra o comportamento dinâmico e a interação em tempo real entre as três camadas descritas anteriormente.

O processo inicia-se de forma assíncrona, com o servidor Scala e a aplicação Flutter a estabelecerem as respetivas subscrições. A partir desse momento, o sistema entra num ciclo contínuo onde a telemetria publicada pelo simulador é consumida pelo *Back-end*.

Tirando partido da concorrência nativa da biblioteca ZIO (*Fibers*), o servidor persiste o registo na base de dados em simultâneo com a emissão do evento para o *Front-end*, garantindo que a interface do utilizador é atualizada instantaneamente.

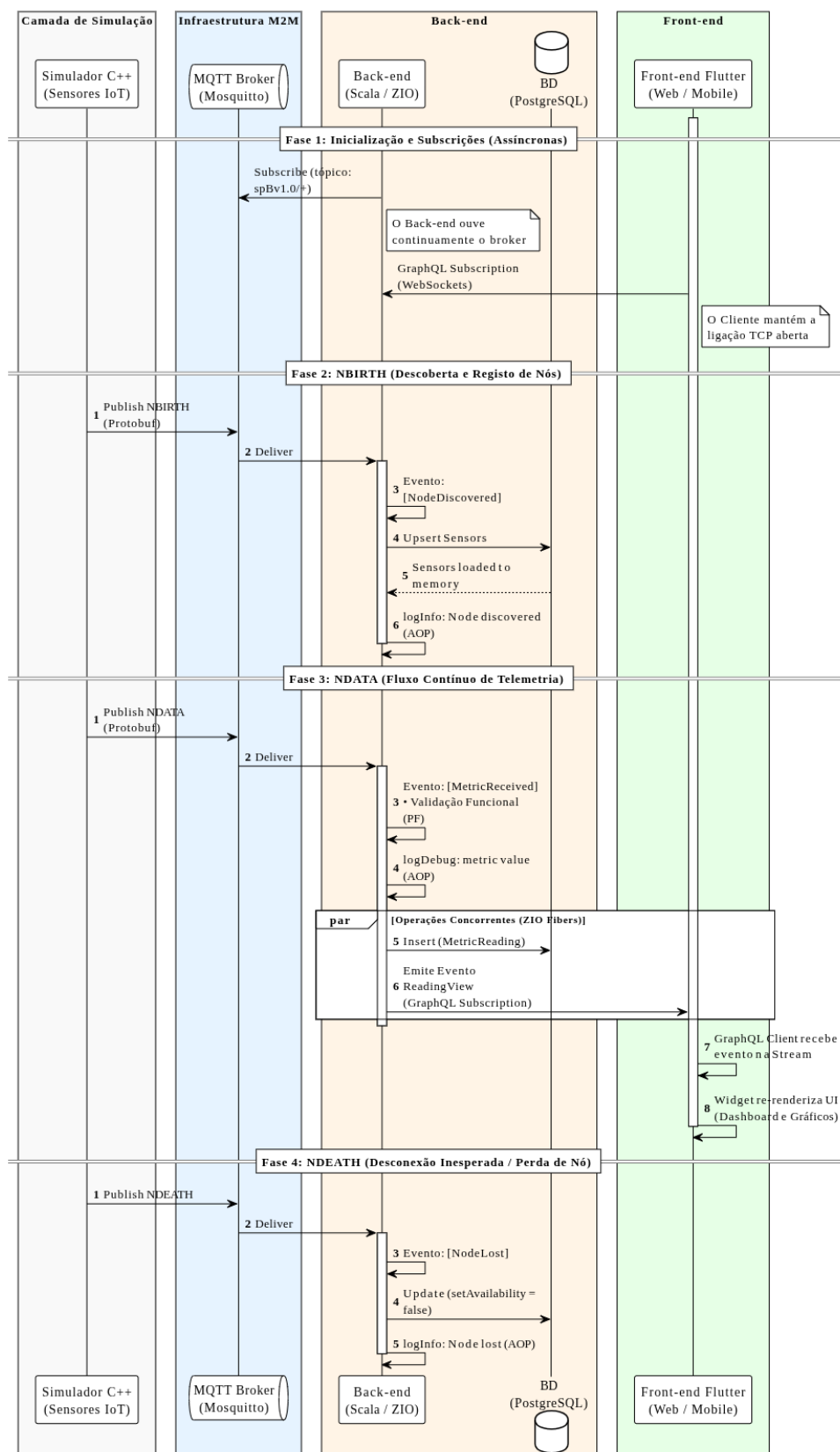


Figura 2: Diagrama de Sequência ilustrando o fluxo de eventos em tempo real.

4 Paradigmas Implementados

Esta secção resume quais os paradigmas utilizados na programação do projeto, onde foram implementados, e como foram programados.

4.1 Programação Funcional

O back-end em *Scala* foi desenvolvido sobre a biblioteca *ZIO HTTP*. A framework *ZIO*, que serve de base para o *ZIO HTTP*, tem como principal característica a forte dependência do paradigma funcional. Neste projeto, a programação funcional não foi apenas uma escolha de sintaxe, mas sim a base para garantir concorrência segura, tratamento de erros, previsibilidade no fluxo de dados e o cumprimento de um dos requisitos do enunciado.

A implementação deste paradigma no back-end manifestou-se através dos seguintes princípios:

- **Imutabilidade e Funções Puras:** Em vez de depender de variáveis de estado que mudam ao longo do tempo, o projeto utilizou estruturas de dados imutáveis (como as *case classes* do *Scala*) para modelar a informação. Isto significa que quando um novo dado de um sensor é processado, o estado anterior não é alterado; em vez disso, é gerada uma nova representação dos dados. As funções desenvolvidas procuraram ser "puras", ou seja, para os mesmos argumentos de entrada, produzem sempre o mesmo resultado sem causar efeitos colaterais inesperados noutras partes do sistema.
- **Efeitos como Valores (*Effects as Values*):** No paradigma imperativo tradicional, uma chamada à base de dados ou a leitura de um *WebSocket* acontece imediatamente. No paradigma funcional implementado com *ZIO*, qualquer operação que interaja com o mundo exterior (operações de I/O, pedidos HTTP) foi modelada como um valor do tipo `ZIO[R, E, A]`. Isto significa que as rotas HTTP e as subscrições de dados não executam ações imediatamente, mas sim constroem uma "receita" ou descrição do programa. A execução destes efeitos é delegada para o final do fluxo, garantindo um controlo total sobre falhas e recursos.

No *front-end*, embora a linguagem *Dart* seja essencialmente orientada a objetos, foram também aplicados princípios de programação funcional. Esta abordagem foi utilizada na

camada dos repositórios para processar de forma reativa os fluxos de dados (*streams*) contínuos provenientes das *subscriptions* do *GraphQL*.

```
1  // returns a stream that can be easily read by the viewmodel
2  return client
3    .subscribe(
4      SubscriptionOptions(
5        document: gql(subscription),
6        variables: {'sensorId': sensorId},
7      ),
8    )
9  // Block every result that is not from the passed sensorId
10 .where((QueryResult result) {
11   if (result.hasException) return false;
12   final data = result.data?['liveReadings'];
13   if (data == null) return false;
14   return data['sensorId'] == sensorId;
15 })
16 .map((QueryResult result) {
17   final r = result.data!['liveReadings'];
18   return SensorData(
19     DateTime.fromMillisecondsSinceEpoch(r['timestamp']),
20     (r['value'] as num).toDouble(),
21   );
22 });
```

Excerto 1: Processamento funcional de uma *Stream* de dados no *front-end*

Conforme ilustrado no excerto de código 1, a manipulação dos eventos em tempo real é construída através de uma *pipeline* declarativa focada em *higher-order functions*. Em vez de recorrer a estruturas de controlo imperativas, o fluxo utiliza o método **where** para filtrar os eventos e o método **map** para projetar a resposta bruta num novo objeto imutável do tipo **SensorData**, garantindo uma transformação de dados pura e sem efeitos colaterais.

4.2 Programação Orientada a Aspectos

A programação orientada a aspetos (AOP) foi utilizada no back-end com o objetivo de isolar a lógica de monitorização e logging do fluxo principal de negócio. Esta funcionalidade

dade foi implementada através de ZIO Aspects, uma funcionalidade nativa da biblioteca ZIO que permite intercepar e modificar o comportamento de efeitos sem alterar a lógica original.

Num ambiente real de recolha de dados, a monitorização contínua do sistema é vital. O registo das métricas através deste aspeto não serve para testar a ligação de rede, mas sim para auditar a performance interna do servidor. Ao medir o tempo exato de execução das operações, é possível detetar estrangulamentos (como lentidão a inserir dados na base de dados) e identificar operações que falharam de forma isolada, tudo isto mantendo o código principal limpo e focado no processamento de dados. Os principais ficheiros que suportam esta funcionalidade são:

- **LoggingAspect:** É o ficheiro central onde o aspeto é definido. Através da função `timed`, o aspeto intercepa a execução, calcula o tempo decorrido, emite alertas de log caso a operação demore mais do que o limite aceitável estabelecido (200 milissegundos) ou emite erros caso a operação falhe.
- **SparkplugEventHandler:** É onde o aspeto é aplicado para monitorizar a entrada de dados. Ele regista o tempo que o sistema demora a decodificar as cargas úteis (payloads) do protocolo MQTT e a encaminhar as mensagens mediante o seu tipo (como o registo de novos nós ou a entrada de métricas).
- **SparkplugMessageProcessor:** O aspeto é aplicado neste pré-processador de eventos para medir a duração de operações críticas do ciclo de vida dos dados, nomeadamente o tempo necessário para guardar uma medição no repositório e para a transmitir (broadcast) aos clientes conectados.
- **InMemoryNodeRegistry:** Neste ficheiro, o aspeto serve para medir o tempo das operações mantidas na memória do sistema. Ele cronometra o tempo exato que o servidor demora a adicionar novos sensores, a procurar um sensor pelo nome, e a apagar sensores que se desconectaram.

4.3 Programação Orientada a Eventos

A programação orientada a eventos foi utilizada no *back-end* para gerir o fluxo contínuo dos dados dos sensores. Embora o servidor dependa do *broker* MQTT, que atua como o

motor externo responsável pela receção e distribuição das mensagens na rede, a aplicação foi desenvolvida para reagir a estes eventos de forma assíncrona e não bloqueante.

O fluxo de eventos da aplicação funciona como uma linha de montagem reativa, baseada em *ZIO Streams*, dividindo-se nas seguintes etapas:

1. **Receção e Filas:** O *back-end* subscreve os tópicos do *broker* através do `MqttSubscriber`.

Assim que uma nova mensagem chega, ela é imediatamente colocada numa fila de espera interna (*Queue*). Permitindo com que exista desacoplamento, pois quem recebe as mensagens não fica bloqueado à espera de quem as processa.

2. **Transformação:** O componente `SparkplugMessageProcessor` consome as mensagens dessa fila e atua como um tradutor. Ele converte mensagens MQTT em bruto para "Eventos de Domínio" compreensíveis pela aplicação (os `SparkplugEvent`), como a descoberta de um novo nó (*NodeDiscovered*) ou a receção de uma nova leitura (*MetricReceived*).

3. **Processamento e Distribuição (*Pub/Sub*):** O `SparkplugEventHandler` com base nestes eventos, guarda a informação na base de dados e, de seguida, publica o evento (`MetricBroadcast`).

No *front-end*, o paradigma orientado a eventos também é utilizado. Através das *subscriptions* do *GraphQL*, tanto a aplicação *web* como *mobile* ficam permanentemente à espera de dados.

Em vez de a aplicação perguntar repetidamente ao servidor se há novos dados (*polling*), o *front-end* atua de forma reativa. Sempre que o *Hub* do servidor emite um novo evento de leitura, este é enviado para a aplicação. O *viewmodel* interceta este evento e notifica a interface gráfica, fazendo com que os gráficos atualizem de forma imediata e automática.

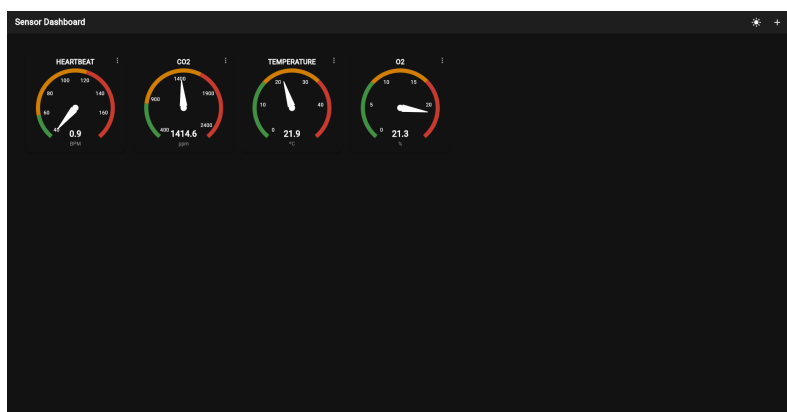
5 Interface de Utilizador

Esta secção apresenta a interface de utilizador desenvolvida em Flutter, mostrando o fluxo principal e a materialização das funcionalidades do sistema.

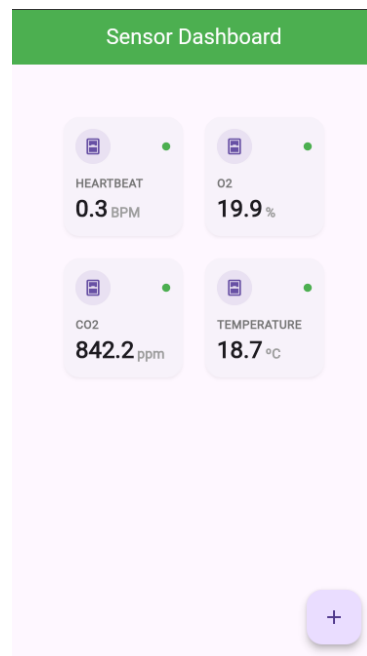
5.1 Ecrã Inicial

O ecrã inicial atua como o ponto central da aplicação, onde o utilizador pode monitorizar em tempo real o estado dos sensores ativos.

Na versão *Web*, presente na Figura 3a, o espaço amplo permite a utilização de gráficos radiais (*gauges*), dando um aspeto industrial adequado a painéis de controlo. Na versão *Mobile* (Figura 3b), a interface foi adaptada para cartões compactos, otimizando o espaço vertical do ecrã do telemóvel. Em termos de integração com o *back-end*, assim que este ecrã é carregado, o *front-end* inicia uma *Subscription* GraphQL (via *WebSockets*) para cada sensor listado, permitindo que qualquer nova medição processada pelo servidor seja imediatamente refletida na interface, sem necessidade de atualizar a página ou fazer pedidos repetitivos (*polling*).



(a) Interface Web



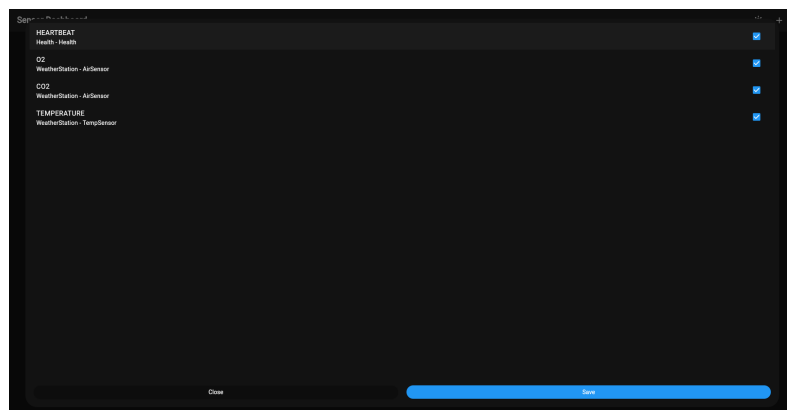
(b) Interface Mobile

Figura 3: Ecrã inicial com as medições dos sensores em tempo real.

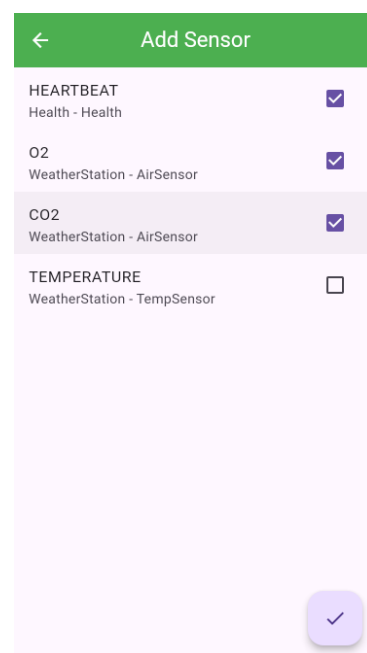
5.2 Ecrã para inserção de novos sensores

Para adicionar novos sensores ao painel principal, o utilizador acede a um menu de seleção. Ao abrir essa interface, a aplicação executa uma *Query GraphQL* ao *back-end* para obter a lista atualizada de todos os nós e sensores registados na base de dados.

Devido às diferenças de navegação entre plataformas, na versão *Mobile* (Figura 4b) esta funcionalidade é apresentada num ecrã dedicado, para haver uma área de toque maior. Por outro lado, na versão *Web* (Figura 4a), a lista é apresentada sob a forma de uma janela sobreposta (*Dialog*). Ao guardar a seleção, as escolhas são persistidas localmente no dispositivo e as respetivas *subscriptions* de tempo real são ativadas no ecrã inicial.



(a) Interface Web



(b) Interface Mobile

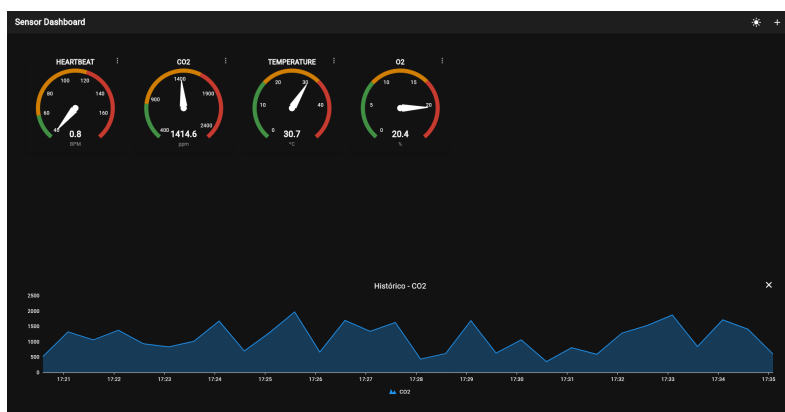
Figura 4: Interface para adição e gestão de sensores no painel.

5.3 Gráfico histórico de medições

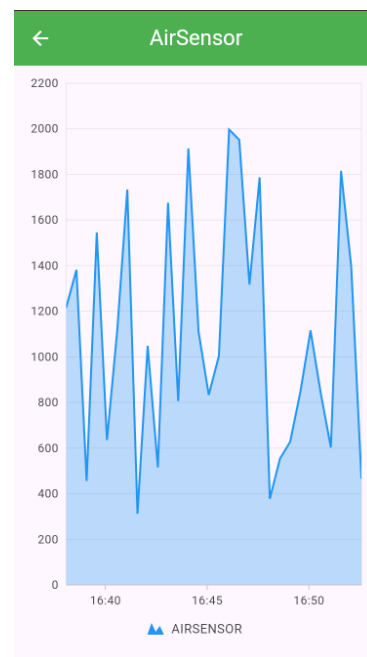
A análise temporal dos dados é crucial para identificar tendências e anomalias. Ao selecionar um sensor específico, o utilizador tem acesso a um gráfico de linhas que ilustra o histórico recente de medições.

No gráfico, a integração com o servidor ocorre em duas fases simultâneas: primeiro, uma *Query GraphQL* é realizada para carregar o histórico de dados guardados na base de dados, de modo a desenhar a linha inicial; de seguida, os dados provenientes da *subscription* em tempo real são injetados diretamente no gráfico, fazendo com que a linha

avance dinamicamente. Na versão *Web* (Figura 5a), este gráfico é exibido através de um painel lateral (*Side Panel*). Na aplicação *Mobile* (Figura 5b), devido às limitações físicas do ecrã, o gráfico é exibido num novo ecrã.



(a) Interface Web



(b) Interface Mobile

Figura 5: Gráfico com o histórico temporal de medições de um sensor selecionado.

5.4 Alterar mínimos e máximos

Como referido na Secção 5.1, o *dashboard Web* utiliza gráficos radiais (*Gauges*), que necessitam de limites pré-definidos para contextualizar adequadamente se as medições estão dentro de um intervalo admissível.

Embora os sensores recebam valores genéricos ao serem adicionados, o utilizador pode ajustá-los através de uma janela sobreposta (*Dialog*), acessível no canto superior direito de cada mostrador (Figura 6). Esta parametrização ajusta dinamicamente a escala do gráfico, permitindo focar a monitorização num intervalo de valores mais exato.

Adicionalmente, este painel permite a remoção rápida do sensor. Todas as alterações efetuadas são guardadas de forma persistente através das preferências locais (*SharedPreferences*), assegurando que as configurações visuais não se perdem ao reiniciar a aplicação.

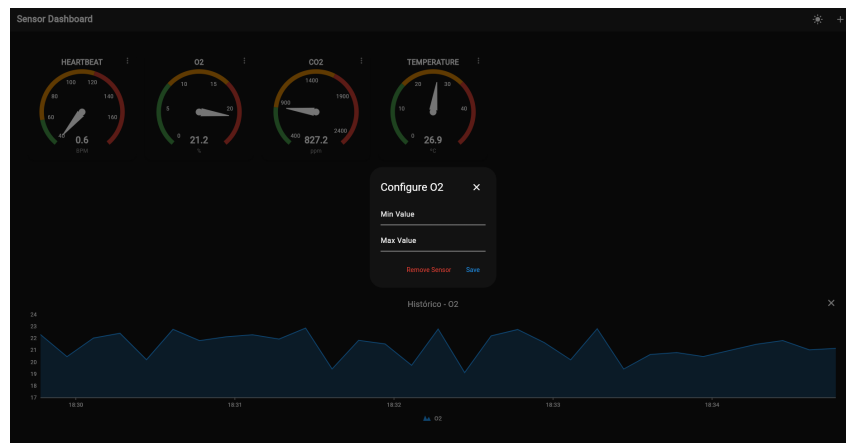


Figura 6: Dialog para edição dos limites do mostrador e remoção do sensor.

6 Metodologia de Avaliação e Desempenho Individual

A atribuição de responsabilidades no decorrer deste projeto ocorreu de forma orgânica. Após uma fase inicial de investigação e exploração das tecnologias adotadas, a divisão de tarefas consolidou-se naturalmente: o desenvolvimento da aplicação cliente (*front-end*), que ficou a cargo do Hugo, e a arquitetura e desenvolvimento do servidor (*back-end*), assumida pelo Diogo.

6.1 Critérios de Atribuição de *Story Points*

A atribuição de *Story Points* foi realizada com base na escala de Fibonacci. Esta métrica avaliou a complexidade técnica, o número de operações envolvidas e as integrações necessárias de cada tarefa de desenvolvimento:

- **1 a 3 pontos (Complexidade Baixa a Média-Baixa):** Tarefas com requisitos bem definidos, configurações base (como a injeção de dependências ou estrutura inicial da aplicação), definição da API GraphQL ou parametrizações simples na interface gráfica (como definir limites para os gráficos radiais).
- **5 pontos (Complexidade Média):** Tarefas com integrações entre múltiplos componentes do sistema, construção do *Dashboard* interativo ou configuração das pontes MQTT e GraphQL *Subscriptions*.
- **8 pontos (Complexidade Alta):** Tarefas tecnicamente desafiantes, exigindo manipulação assíncrona de eventos, orquestração contínua do simulador de dados e renderização de séries temporais reativas combinadas com histórico.

6.2 Distribuição do Esforço e Contribuição

Após a atribuição e somatório dos *Story Points* (SP) de todas as fases do projeto, apurou-se um total de 68 SP. A Figura 7 ilustra a distribuição global do esforço:

6.2.1 Contribuição: Diogo (52.8% – 94 SP)

O Diogo assumiu a responsabilidade pela arquitetura e desenvolvimento do *back-end* em *Scala* utilizando o ecossistema *ZIO*. A sua carga de trabalho esteve concentrada no processamento não-bloqueante de dados e na infraestrutura reativa:

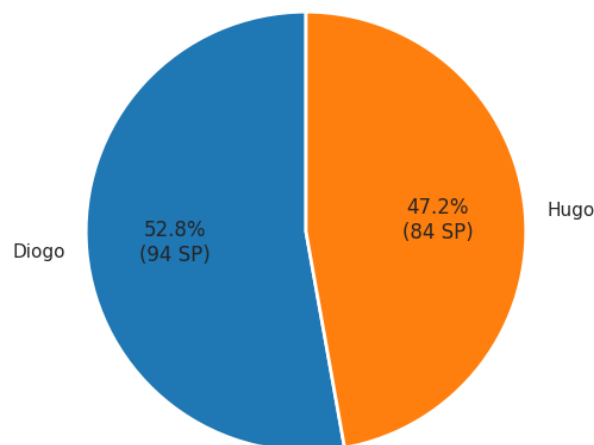


Figura 7: Distribuição da carga de trabalho do projeto com base em *Story Points*.

- **Simulador e Integração MQTT:** A tarefa mais exigente do *back-end* (avaliada em 8 SP), que envolveu a integração e orquestração do simulador de sensores IoT, garantindo o correto fluxo de dados em formato *Sparkplug B* através do *broker* MQTT.
- **Processamento de Eventos:** Tarefas de média-alta complexidade relativas ao processamento em memória do estado dos nós (*Nodes*) e telemetrias (*Metrics*), recorrendo à programação orientada a eventos.
- **Motor GraphQL e Base de Dados:** Configuração da infraestrutura persistente (*PostgreSQL*) e implementação da API GraphQL (com recurso à biblioteca *Caliban*), culminando na gestão das *subscriptions* reativas.

6.2.2 Contribuição: Hugo (47.2% – 84 SP)

O Hugo assumiu a estruturação, lógica e interface visual do cliente interativo (*front-end*) em *Flutter*. A sua contribuição focou-se no domínio de apresentação reativa, sincronização de estado (*MVVM*) e componentes visuais de análise:

- **Gráficos e Séries Temporais:** A tarefa de maior exigência técnica no cliente, que consistiu em fundir dados históricos com eventos em tempo real provenientes das *subscriptions*, traduzindo-os em gráficos de linhas dinâmicos (*Time Series Charts*).

- **Sincronismo em Tempo Real:** Configuração complexa do cliente GraphQL no lado da aplicação móvel/web, gerindo as sessões *WebSocket* para a atualização contínua de dados.
- **Dashboard Visual:** O desenvolvimento do ecrã de controlo, que incluiu a renderização dos mostradores radiais (*Gauges*), permitindo ainda configurações locais personalizadas de limites mínimos e máximos.

7 Conclusão

O presente relatório documenta a criação de um sistema reativo para monitorização de sensores IoT, destacando-se pela separação de responsabilidades entre o *back-end* e o *front-end*.

No *back-end*, desenvolvido em *Scala* com o ecossistema *ZIO*, implementou-se uma arquitetura não bloqueante orientada a eventos (*Event-Driven*), integrada com um *broker* MQTT (*Sparkplug B*). Simultaneamente, a Programação Orientada a Aspectos (AOP) que permitiu isolar o registo de métricas (*logging*), mantendo a lógica de negócio modular.

No *front-end*, o *Flutter* permitiu gerar interfaces *Web* e *Mobile* a partir de uma base de código única. A conjugação do padrão MVVM com *subscriptions GraphQL* (*WebSockets*) assegurou um fluxo de comunicação eficiente, refletindo as leituras dos sensores nos mostradores gráficos em tempo real e com latência impercetível.

7.1 Trabalho Futuro

Embora os requisitos propostos para o sistema tenham sido plenamente cumpridos, o ciclo de desenvolvimento de *software* é iterativo. Dessa forma, identificam-se as seguintes melhorias potenciais para a evolução contínua da plataforma:

- **Sistema de Alertas e Notificações:** Aproveitar os limites mínimos e máximos já definidos no *front-end* para criar um mecanismo de atuação no servidor, capaz de disparar notificações *push* ou alertas por email sempre que um sensor registre valores anómalos.
- **Segurança e Autenticação:** Integrar um sistema de autenticação e autorização (como JWT ou *OAuth*) para garantir que apenas operadores credenciados têm acesso ao painel de monitorização e à *API GraphQL*.